

Learning MPC Dynamics and Neural Lyapunov Certificates for Drones

Aniket Bhosale

*Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
aniketbh@andrew.cmu.edu*

Avi Dube

*Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
avid@andrew.cmu.edu*

Abstract—Quadrotor control requires models and controllers that can handle strongly nonlinear dynamics while remaining fast enough for real-time deployment and reliable enough to avoid unsafe behavior. In this project, we study a learning-augmented model predictive control pipeline for drone stabilization and tracking in a high-fidelity MuJoCo simulator. Our approach combines a physics-based quadrotor model with a learned neural residual model that captures unmodeled aerodynamic effects and other nonlinearities, enabling more accurate prediction inside the MPC optimization loop. The controller is implemented as a nonlinear MPC problem in CasADi with IPOPT, using costs on state error, control effort, and input smoothness together with a strong terminal penalty to encourage stable hover. To analyze safety and stability, we additionally train a generalized neural Lyapunov certificate around the hover regime using rollout data from the MPC policy. This learned certificate provides a data-driven way to assess whether the controller admits a locally stabilizing energy-like function even when the closed-loop policy is not analytically tractable.

Index Terms—quadrotors, model predictive control, system identification, neural dynamics, Lyapunov stability

I. INTRODUCTION

Quadrotor drones are attractive platforms for robotics because they are agile, inexpensive, and capable of operating in cluttered environments. At the same time, they are difficult to control in practice. Their dynamics are nonlinear, coupled across translational and rotational motion, and strongly affected by aerodynamic disturbances, modeling error, and noisy sensing. For small aerial platforms in particular, even modest mismatch between the assumed dynamics and the true system can produce degraded tracking performance, oscillatory behavior, or loss of stability near hover.

Model Predictive Control (MPC) is a natural framework for this setting because it repeatedly solves a finite-horizon optimal control problem while explicitly accounting for dynamics and control constraints. For drones, MPC is appealing because it can trade off position regulation, velocity damping, and control effort in a principled manner. However, the quality of an MPC controller depends heavily on the fidelity of its predictive model. A purely analytical model is often too crude to capture aerodynamic effects and other nonlinearities, while a purely learned black-box model can be difficult to differentiate, hard to generalize outside the training distribution, and challenging to trust in safety-critical regimes.

This project explores a hybrid alternative that combines first-principles structure with learned corrections. We build a high-fidelity MuJoCo simulation environment for a Crazyflie-style quadrotor with rigid-body dynamics, aerodynamic effects, injected stochastic disturbances, and domain randomization to reduce overfitting to a single simulator instance. Using rollout data from this environment, we learn residual dynamics on top of a known physics-based model. The resulting hybrid model is then embedded inside a nonlinear MPC controller solved with CasADi and IPOPT. The cost function penalizes deviation from a desired target state, excessive control effort, and rapid changes in control inputs, with additional terminal weighting to promote hover stability.

Beyond performance, we investigate whether the learned closed-loop behavior is stable near the operating point of interest. Because the controller is obtained through optimization with learned components, deriving a classical analytic Lyapunov proof is difficult. To address this challenge, we train a generalized neural Lyapunov certificate around the hover region using data collected from the MPC policy. The certificate is parameterized through a learned shaping function together with residual components that capture the local closed-loop error dynamics. This provides a practical, data-driven mechanism for assessing local stability and safety properties of the controller.

The central goal of the project is therefore twofold: first, to improve drone control performance by learning residual dynamics that enhance MPC prediction accuracy, and second, to study whether neural Lyapunov methods can provide meaningful post hoc stability guarantees for the resulting controller. Taken together, these components form a pipeline that links simulation, model learning, optimal control, and stability certification for nonlinear aerial systems.

A. Project Milestones

As part of our work, we also identify some key milestones that we address during our project.

- 1) **Simulation Environment Live.** Stand up the MuJoCo quadrotor environment with domain randomization and collect rollout transitions.
- 2) **Dynamics Model Ready.** Train the residual dynamics model f_θ on the rollout simulated transitions.

- 3) **Learned MPC Running.** Embed f_θ symbolically into CasADi and solve the resulting NLP with IPOPT, then validate the closed-loop controller in simulation.
- 4) **Closed-Loop Simulation Validated.** Integrate the MPC controller into the simulation, run test episodes, and visualize the resulting trajectories.
- 5) **Lyapunov Certificate Verified.** Train and verify the neural Lyapunov certificate on real or transferred rollout data.

We also identified a stretch milestone of:

Real Rollouts Collected. Deploy the learned MPC on a physical drone and collect rollouts.

II. PROBLEM FORMULATION

In order to formulate our problem, we can decompose it into three different stages.

A. Estimating System Dynamics

In our work, we consider the problem of learning a predictive model for a quadrotor system with nonlinear dynamics. The system evolves in discrete time according to

$$x_{k+1} = f(x_k, u_k), \quad (1)$$

where $x_k \in \mathbb{R}^{12}$ denotes the system state and $u_k \in \mathbb{R}^4$ denotes the control input.

The state is defined as

$$x = [p, \eta, v, \omega], \quad (2)$$

where $p \in \mathbb{R}^3$ is the position, $\eta = [\phi, \theta, \psi] \in \mathbb{R}^3$ are the Euler angles, $v \in \mathbb{R}^3$ is the linear velocity, and $\omega \in \mathbb{R}^3$ are the body angular rates. The control input is given by

$$u = [T, \tau_x, \tau_y, \tau_z], \quad (3)$$

representing total thrust and body torques.

We assume access to a dataset of transitions

$$\mathcal{D} = \left\{ \left(x_k^{(i)}, u_k^{(i)}, x_{k+1}^{(i)} \right) \right\}_{i=1}^N, \quad (4)$$

and seek to learn a model $\hat{f}$ that approximates the true system dynamics.

To incorporate prior knowledge, we decompose the dynamics into a physics-based component and a learned residual:

$$x_{k+1} = f_{\text{phys}}(x_k, u_k) + \delta_\theta(x_k, u_k), \quad (5)$$

where f_{phys} is derived from first-principles dynamics and δ_θ is a neural network parameterized by θ .

The physics-based model is defined through a continuous-time nonlinear ordinary differential equation:

$$\dot{x} = f_{\text{ode}}(x, u), \quad (6)$$

where f_{ode} captures rigid-body translational and rotational dynamics, including gravity, thrust, full rotation kinematics, and gyroscopic coupling.

The discrete-time physics model is obtained by numerically integrating the ODE using a fourth-order Runge–Kutta (RK4) scheme:

$$f_{\text{phys}}(x_k, u_k) = \text{RK4}(f_{\text{ode}}, x_k, u_k, \Delta t). \quad (7)$$

The residual term $\delta_\theta(x_k, u_k)$ accounts for unmodeled effects such as aerodynamic drag, rotor wake interactions, and actuator dynamics.

The model parameters are learned by minimizing the prediction error over the dataset:

$$\min_{\theta} \mathbb{E}_{(x, u, x') \sim \mathcal{D}} \left[\|x' - (f_{\text{phys}}(x, u) + \delta_\theta(x, u))\|^2 \right]. \quad (8)$$

B. MPC Control Problem

We consider the problem of controlling the quadrotor to track a desired reference state x_{ref} using a finite-horizon optimal control formulation.

At each time step k , we solve the following nonlinear optimization problem over a prediction horizon of length N :

$$\begin{aligned} \min_{\{x_k, u_k\}_{k=0}^N} \quad & \sum_{k=0}^{N-1} \left[(x_k - x_{\text{ref}})^\top Q (x_k - x_{\text{ref}}) \right. \\ & + (u_k - u_{\text{hover}})^\top R (u_k - u_{\text{hover}}) \\ & + (u_k - u_{k-1})^\top R_\Delta (u_k - u_{k-1}) \\ & \left. + (x_N - x_{\text{ref}})^\top Q_f (x_N - x_{\text{ref}}) \right] \end{aligned} \quad (9)$$

subject to the hybrid system dynamics described in (5) and the initial condition:

$$x_0 = \hat{x}, \quad (10)$$

where $\hat{x}$ is the current measured state.

The matrices $Q \succeq 0$, $Q_f \succeq 0$, $R \succ 0$, and $R_\Delta \succ 0$ are weighting matrices that penalize state tracking error, control effort, and control rate variation, respectively. The term u_{hover} denotes the nominal control input required for hover.

Control inputs are constrained to lie within admissible bounds:

$$u_{\min} \leq u_k \leq u_{\max}, \quad \forall k. \quad (11)$$

C. Lyapunov Stability Analysis

To analyze the closed-loop behavior of the nonlinear model predictive controller under hybrid learned dynamics, we adopt a Lyapunov-based certification framework for finite-horizon optimal control policies.

Let the tracking error be defined as

$$e_k = x_k - x_{\text{ref}}. \quad (12)$$

The receding-horizon MPC policy induces an implicit value function

$$V_\pi(e_k) = \sum_{i=0}^{N-1} \ell(e_{k+i}, u_{k+i}) + \ell_f(e_{k+N}), \quad (13)$$

where $\ell(\cdot)$ is the stage cost defined by the MPC objective and $\ell_f(\cdot)$ is the terminal cost. Under standard positive definiteness assumptions on $Q \succeq 0$, $Q_f \succeq 0$, and $R \succ 0$, $V_\pi(e)$ serves as a nominal Lyapunov candidate for the MPC-controlled system.

However, due to nonlinear hybrid dynamics in (5) and approximation errors introduced by the learned residual model δ_θ , classical one-step decrease conditions of the form

$$V_\pi(e_{k+1}) - V_\pi(e_k) \leq 0 \quad (14)$$

do not hold globally.

To address this limitation, we adopt the generalized Lyapunov framework introduced in [1] based on finite-horizon weighted decrease conditions.

1) *Generalized Lyapunov Function*: We define the Lyapunov candidate

$$V(e) = e^\top Q_f e + \phi_{\theta_1}(\tilde{e}) - \phi_{\theta_1}(0) + \beta \|e\|^2, \quad (15)$$

where e is the tracking error defines in (14), $\tilde{e} = e \otimes s$ is a normalized state using scaling vector s , $\phi_{\theta_1}(\cdot)$ is a neural residual correction term, and $\beta \|e\|^2$ enforces strict positivity in a neighborhood of the origin. This construction ensures $V(e) \geq 0$ and $V(0) = 0$, while preserving consistency with the MPC terminal cost through the quadratic term $e^\top Q_f e$.

2) *Finite-Horizon Decrease Condition*: Instead of enforcing monotonic decrease at a single time step, we define a generalized multi-step Lyapunov condition over a prediction horizon of length M :

$$F(e_k) = \frac{1}{M} \sum_{i=1}^M \sigma_i(e_k) V(e_{k+i}) - (1 - \bar{\alpha}) V(e_k), \quad (16)$$

where $\sigma_i(e_k) \geq 0$ are state-dependent weighting coefficients, $\sum_{i=1}^M \sigma_i(e_k) = M$, and $\bar{\alpha} \in (0, 1)$ is a relaxation parameter. The function $\sigma(e_k)$ redistributes Lyapunov decrease requirements across the horizon, allowing delayed contraction behavior typical in underactuated nonlinear systems. A state e_k is certified as stable if

$$F(e_k) \leq 0. \quad (17)$$

3) *Role of Learned Components*: The learned functions ϕ_{θ_1} and σ_{θ_2} play complementary roles. The residual Lyapunov correction ϕ_{θ_1} models deviations between the MPC-induced value function and a valid Lyapunov function for the true hybrid system. The temporal weighting σ_{θ_2} allocates importance across the prediction horizon, enabling the certificate to emphasize regions where contraction is most reliable.

4) *Certification Objective*: The parameters (θ_1, θ_2) are obtained by solving

$$\min_{\theta_1, \theta_2} \mathbb{E}_{e \sim \mathcal{D}} [\max(0, F(e))], \quad (18)$$

which penalizes violations of the generalized Lyapunov decrease condition. A valid certificate is obtained when

$$F(e) \leq 0, \quad \forall e \in \mathcal{D}_{\text{test}}. \quad (19)$$

III. SIMULATION SETUP

We evaluate the proposed controller in a high-fidelity MuJoCo simulation built around a Crazyfly-style quadrotor model. The simulator captures rigid-body translational and rotational dynamics and is configured with fluid properties so

that external wind disturbances produce physically meaningful aerodynamic effects on the vehicle. In the demonstration environment, the air density and viscosity are set explicitly and wind is injected directly through the MuJoCo dynamics options, which allows the same model to be tested under a range of disturbance magnitudes and directions.

The control state follows a 12-dimensional representation consisting of position, Euler orientation, linear velocity, and angular velocity. The nominal task is hover stabilization at a fixed reference pose, with the target position set to $(x, y, z) = (0, 0, 1.2)$ meters and zero desired attitude and velocity. Each rollout is initialized close to the ground, after which the controller regulates the vehicle to the hover point while compensating for modeling error and disturbances. The MPC loop runs at a control period of 0.02 seconds, corresponding to a 50 Hz replanning rate, while the MuJoCo simulator integrates the underlying dynamics at its internal physics timestep.

To make the controller output compatible with the simulator, the optimized wrench command is mapped to four rotor thrusts using the quadrotor geometry and a yaw-torque coefficient. Rotor commands are clipped to a feasible thrust range before being applied to the MuJoCo actuators. This setup provides a clean interface between the hybrid dynamics model used by the MPC solver and the lower-level actuation model required by the simulator.

The simulation environment is also designed to support robustness analysis. In particular, wind can be varied independently along the three Cartesian axes during a rollout, enabling direct stress-testing of hover performance under persistent disturbances. The demo interface additionally exposes reset and telemetry functionality so that position error, vehicle position, and simulation time can be monitored online while the controller is running. Together, these design choices create a controlled but realistic environment for evaluating both trajectory regulation performance and the stability behavior of the learned MPC policy.

IV. METHODOLOGY

Our pipeline consists of four interconnected stages: a high-fidelity simulation environment, residual dynamics learning, a nonlinear model predictive controller (NMPC), and a post-hoc neural Lyapunov certificate. We describe each below.

A. Simulator and Data Collection

We simulate a Crazyfly-class quadrotor in MuJoCo at an internal physics timestep significantly smaller than the control period $\Delta t = 0.02$ s (50 Hz). The vehicle is modeled as a rigid body with mass m and diagonal inertia $\text{diag}(I_x, I_y, I_z)$ extracted directly from the MuJoCo XML, ensuring the analytical physics model and the simulator share identical parameters. Four rotor actuators produce thrust at the body frame; we map the optimized wrench command $u = [T, \tau_x, \tau_y, \tau_z]$ to individual rotor thrusts $u_r \in \mathbb{R}^4$ using

$$u_r = M_{\text{mix}}^{-1} u, \quad (20)$$

where M_{mix} is determined by the arm length ℓ and yaw-torque coefficient k of the quadrotor geometry. Rotor commands are clipped to a feasible range before being applied.

To collect training data, we implement a baseline linear-MPC controller that uses a small-angle, hover-linearized model of the quadrotor dynamics. Linearizing around the hover equilibrium $(p, \eta, v, \omega) = (p_{\text{ref}}, 0, 0, 0)$ with nominal thrust $T_0 = mg$ and zero torque, and discretizing with forward Euler at sample time Δt , gives

$$x_{k+1} = A_d x_k + B_d u_k + c, \quad (21)$$

where $A_d = I_{12} + A\Delta t$, $B_d = B\Delta t$, and the constant $c \in \mathbb{R}^{12}$ accounts for the gravity offset. The continuous-time matrices A and B encode only the dominant linear couplings:

$$A = \begin{bmatrix} 0_3 & 0_3 & I_3 & 0_3 \\ 0_3 & 0_3 & 0_3 & I_3 \\ 0_3 & A_{v\eta} & 0_3 & 0_3 \\ 0_3 & 0_3 & 0_3 & 0_3 \end{bmatrix}, \quad B = \begin{bmatrix} 0_{3 \times 4} \\ 0_{3 \times 4} \\ B_v \\ B_\omega \end{bmatrix}, \quad (22)$$

where $A_{v\eta}$ encodes the $\dot{v}_x = g\theta$, $\dot{v}_y = -g\phi$ couplings produced by tilting the thrust vector, $B_v = [0, 0, 1/m]^\top$ on the thrust column, and $B_\omega = \text{diag}(1/I_x, 1/I_y, 1/I_z)$ on the torque columns. The linear controller solves a quadratic program at each step with the same state-error, control-effort, and rate-of-change penalties used in the nonlinear formulation, and is solved efficiently by OSQP. To compensate for the steady-state offset introduced by the linearization (which under-predicts the thrust required at nonzero tilt), the controller maintains an integral term on (x, y, z, ψ) tracking error,

$$i_k = \text{clip}\left(i_{k-1} + (x_{\text{ref}} - \hat{x})_{\{x,y,z,\psi\}} \Delta t, \pm i_{\text{max}}\right), \quad (23)$$

which is added as a feedforward correction to u_{hover} .

We run this baseline in closed loop across multiple episodes with diverse hover references and record the transition tuple (x_k, u_k, x_{k+1}) at every control step, where x_k is sampled from the simulator's true state and u_k is the wrench command issued by the controller. Several thousand such transitions are collected across episodes and stored for offline residual training.

B. Residual Dynamics Learning

The first-principles component is derived from the standard rigid-body equations of motion for a quadrotor. Let $R(\eta) \in SO(3)$ denote the ZYX rotation matrix from body to world frame,

$$R(\eta) = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}, \quad (24)$$

where $c_\alpha = \cos \alpha$, $s_\alpha = \sin \alpha$, and let $W(\eta)$ be the Euler-rate Jacobian mapping body angular rates to Euler-angle derivatives,

$$W(\eta) = \begin{bmatrix} 1 & s_\phi t_\theta & c_\phi t_\theta \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi / c_\theta & c_\phi / c_\theta \end{bmatrix}, \quad t_\alpha = \tan \alpha. \quad (25)$$

The continuous-time dynamics $f_{\text{ode}} : \mathbb{R}^{12} \times \mathbb{R}^4 \rightarrow \mathbb{R}^{12}$ take the form

$$\begin{aligned} \dot{p} &= v, \\ \dot{\eta} &= W(\eta) \omega, \\ \dot{v} &= R(\eta) \frac{1}{m} \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}, \\ \dot{\omega} &= I^{-1}(\tau - \omega \times I \omega), \end{aligned} \quad (26)$$

where $I = \text{diag}(I_x, I_y, I_z)$, $\tau = [\tau_x, \tau_y, \tau_z]^\top$, and g is gravitational acceleration. The translational dynamics include the full nonlinear thrust mapping through $R(\eta)$, and the rotational dynamics include the gyroscopic coupling $\omega \times I \omega$.

The discrete-time physics step is obtained by integrating (26) over one control period Δt with a fourth-order Runge-Kutta scheme,

$$\begin{aligned} k_1 &= f_{\text{ode}}(x_k, u_k), \\ k_2 &= f_{\text{ode}}\left(x_k + \frac{\Delta t}{2} k_1, u_k\right), \\ k_3 &= f_{\text{ode}}\left(x_k + \frac{\Delta t}{2} k_2, u_k\right), \\ k_4 &= f_{\text{ode}}(x_k + \Delta t k_3, u_k), \\ f_{\text{phys}}(x_k, u_k) &= x_k + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \quad (27)$$

RK4 is fourth-order accurate in Δt , which lets us use the same $\Delta t = 0.02$ s for both data collection and MPC prediction without significant integration error.

For each transition in the dataset, we compute the residual target

$$\delta_k = x_{k+1} - f_{\text{phys}}(x_k, u_k), \quad (28)$$

which captures everything the analytical model fails to predict.

The residual is parameterized by a small multilayer perceptron $\delta_\theta : \mathbb{R}^{16} \rightarrow \mathbb{R}^{12}$ with three hidden layers of 64 units and SiLU activations. We choose SiLU rather than ReLU for two reasons: (i) aerodynamic effects are smooth functions of state, so a smooth activation is a more appropriate inductive bias; and (ii) SiLU is smooth and easily expressed in CasADi as $\text{SiLU}(x) = x \sigma(x)$, where σ is the sigmoid function, simplifying the symbolic translation needed for use inside the MPC NLP. The output layer is initialized near zero so that $\delta_\theta \approx 0$ at the start of training, and the model only learns nonzero corrections where the data demands them. The number of parameters in the model are also deliberately kept small to ensure quick differentiation and solve times by the MPC solver across the model.

We train δ_θ by minimizing a Huber loss with $\delta = 0.01$ on $(x_k, u_k) \rightarrow \delta_k$ pairs, using AdamW with cosine learning-rate annealing and gradient norm clipping. The Huber loss is preferred to MSE because the dataset occasionally contains large residuals from near-ground contacts and aggressive transients, and we do not want these outliers to dominate the gradient. Inputs are normalized by the per-dimension mean and standard deviation of the training set..

C. Nonlinear MPC with Hybrid Dynamics

Both the physics integrator (27) and the trained residual network are expressed symbolically in CasADi, allowing

IPOPT to differentiate through the entire hybrid dynamics map automatically. The full one-step model

$$\hat{f}(x_k, u_k) = f_{\text{phys}}(x_k, u_k) + \delta_\theta(x_k, u_k) \quad (29)$$

appears as an equality constraint between consecutive shooting variables x_k and x_{k+1} in the NLP.

We use a horizon of $N = 25$ steps (0.5 s lookahead), state weights $Q = \text{diag}(20, 20, 100, 5, 5, 50, 2, 2, 5, 1, 1, 10)$, control weights $R = 0.01 I$, rate-of-change weights $R_\Delta = 0.1 I$, and terminal cost $Q_f = 10 Q$. [2] was referred to when making decisions about the planning horizon and the costs. The control bounds are $T \in [0, 2mg]$, $|\tau_{x,y}| \leq 0.005 \text{ N}\cdot\text{m}$, and $|\tau_z| \leq 0.02 \text{ N}\cdot\text{m}$, which match the physical actuation limits of the simulated platform. Although the linear baseline of the data collection stage uses a longer horizon ($N = 50$) to compensate for its modeling error, the hybrid model's accuracy lets us shorten the horizon to 25 steps without sacrificing tracking performance, which keeps the per-step NLP tractable within the 20 ms control budget.

To improve real-time performance we exploit the receding-horizon structure through warm starting. The solution at step k is shifted by one stage and used as the IPOPT initial guess at step $k + 1$. If IPOPT fails to converge within the allotted iteration limit, the controller falls back to a tilt-compensated hover thrust $u_{\text{fb}} = [mg/(\cos \phi \cos \theta), 0, 0, 0]$, which holds attitude while the next solve is attempted.

D. Generalized Lyapunov Certification

After the NMPC controller is fixed, we estimate a local Lyapunov certificate around the hover equilibrium x_{ref} . Because the closed-loop dynamics are induced by the solution of an NLP with learned components, we cannot derive a certificate analytically and instead train one from data, following the generalized framework of [1].

a) Rollout collection: We sample initial error states $e_0 \sim \text{Unif}([-D, D]^{12})$ from a tight box around hover ($\pm 0.3 \text{ m}$ position, $\pm 11^\circ$ roll/pitch, $\pm 20^\circ$ yaw, $\pm 1 \text{ m/s}$ velocity, $\pm 1 \text{ rad/s}$ angular rate), excluding a small ball $\|e_0\| > \delta$ around the origin since the controller settles to a neighborhood of the equilibrium rather than exactly to it. For each sampled e_0 we simulate $M = 30$ closed-loop steps (0.6 s) using the hybrid dynamics and the same NMPC controller used at deployment, recording the full error trajectory $\{e_0, e_1, \dots, e_M\}$. We collect 5,000 training rollouts and 30,000 test rollouts.

b) Lyapunov candidate: The certificate function takes the form

$$V(e) = e^\top Q_f e + |\phi_{\theta_1}(\tilde{e}) - \phi_{\theta_1}(0)| + \beta \|e\|^2, \quad (30)$$

where $\tilde{e} = e \oslash s$ is the error normalized by the domain half-width s , and $\phi_{\theta_1} : \mathbb{R}^{12} \rightarrow \mathbb{R}$ is a three-layer MLP with 128 tanh units. The terminal cost Q_f is reused from the NMPC objective, which gives V the same local geometry the controller is already optimizing for. The neural term ϕ_{θ_1} shapes V to fit the closed-loop contraction pattern, and $\beta \|e\|^2$ with $\beta = 0.01$ guarantees strict positivity in a neighborhood of the origin. The absolute value around the ϕ difference and

the explicit $\phi(0)$ subtraction together enforce $V(e) \geq 0$ and $V(0) = 0$ by construction.

c) Step weighting: The horizon weighting $\sigma_{\theta_2} : \mathbb{R}^{12} \rightarrow \mathbb{R}_{\geq 0}^M$ is a two-layer tanh MLP with a softmax output rescaled by M , ensuring the constraint $\sum_{i=1}^M \sigma_i = M$ holds exactly without any explicit penalty. Tanh is used in both networks to give globally Lipschitz mappings, which is required by the theoretical results of [1].

d) Training and evaluation: Both networks are trained jointly by minimizing the hinge loss on $F(e)$,

$$\mathcal{L}(\theta_1, \theta_2) = \mathbb{E}_{e_0 \sim \mathcal{D}_{\text{train}}} [\max(0, F(e_0))], \quad (31)$$

with $\bar{\alpha} = 0.02$. We train for 800 epochs with Adam, batch size 256, learning rate 3×10^{-4} , and cosine annealing. The certificate is then evaluated on the held-out test set, and we report the percentage of states satisfying $F(e) \leq 0$, the maximum violation, and the distribution of horizon weights σ_i . A successful certificate must produce nonnegative V , vanishing $V(0)$, monotone growth of V away from the origin, full satisfaction of $F \leq 0$ on test data, and back-loaded σ weights.

V. RESULTS

A. Nonlinear MPC Controller

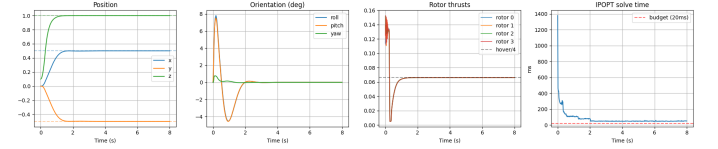


Fig. 1. Closed-loop NMPC hover stabilization with the hybrid physics + residual model. Position, attitude, rotor thrusts, and per-step IPOPT solve time are shown.

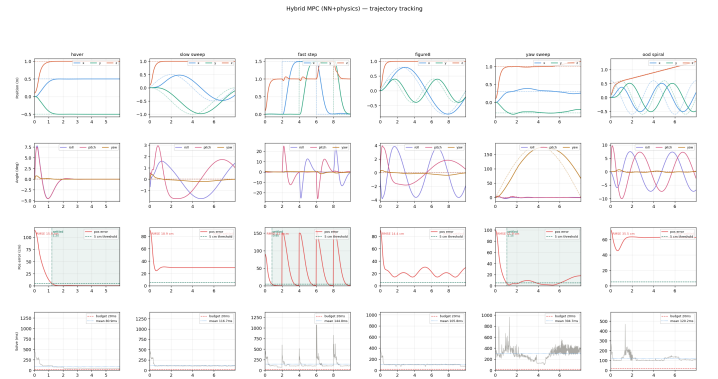


Fig. 2. Benchmark sweeps of the NMPC controller across multiple reference trajectories.

Fig. 1 shows the closed-loop response of the hybrid NMPC when commanded to a hover reference. The controller settles to the target position quickly with no overshoot, holds attitude near zero throughout the maneuver. This confirms that the hybrid dynamics model is accurate enough for the NMPC to

plan trajectories that the simulator faithfully executes, with no observable mismatch between predicted and realized closed-loop behavior.

The benchmark sweeps in Fig. 2 confirm that the controller tracks z -axis references tightly across a range of setpoints. Lateral (x, y) tracking exhibits a small lag, which is expected given the asymmetric weighting in $Q = \text{diag}(20, 20, 100, \dots)$. The position weights deliberately prioritize altitude regulation over lateral tracking, since our goal requires tight vertical hover above all else. Fine-tuning Q would yield better lateral response and overall tracking over general trajectories but is outside the scope of this work.

B. Lyapunov Stability Analysis

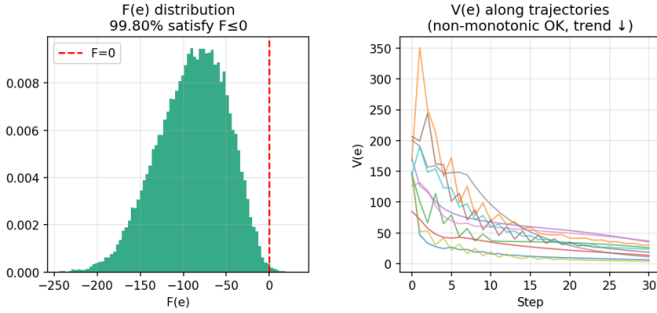


Fig. 3. Generalized Lyapunov certificate diagnostics: training loss, horizon weight concentration σ_i , distribution of $F(e)$ on the test set, and $V(e)$ along sample trajectories.

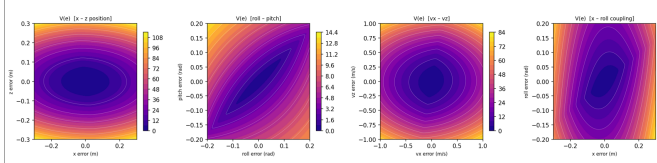


Fig. 4. Two-dimensional slices of the learned Lyapunov function $V(e)$ across position, attitude, velocity, and position-attitude coupling planes.

Fig. 3 reports the test-set diagnostics for the trained certificate. The generalized decrease condition $F(e) \leq 0$ defined in (16) holds on 99.80% of the 30,000 initial error states sampled uniformly from the certified domain. This provides strong empirical evidence that the closed-loop system contracts toward the hover equilibrium throughout the certified region. The $V(e)$ trajectories shown in the same figure exhibit the expected non-monotonic but globally decreasing behavior. Since we loosen the decreasing behavior to be generalized, the brief increases in V are admissible provided the overall trend is decreasing. This is the flexibility of the generalized Lyapunov constraint allows the certificate to succeed on a system whose closed-loop dynamics involve alignment movements that temporarily grow position error.

Fig. 4 visualizes $V(e)$ for better analysis. The translational slices (x - z , v_x - v_z) are smooth and approximately symmetric

about the origin, as expected from the diagonal Q_f obtained from the MPC terminal cost. The attitude slice (roll-pitch) and the cross-coupling slice (x -roll) exhibit notably tighter level sets, reflecting the heavy attitude weighting in Q_f . This is consistent with the underlying control objective since attitude errors propagate quickly into position errors through the rotated thrust vector, and a controller that hopes to stabilize hover must suppress them aggressively.

VI. CONCLUSION

In this work, we presented a learning-based nonlinear model predictive control pipeline for quadrotor hover stabilization, paired with a post-hoc neural Lyapunov certificate. As part of the pipeline, we first build a linear MPC controller using a small-angle, hover-linearized model of the quadrotor and run it in closed loop within the high-fidelity MuJoCo simulation environment to collect rollout data. Using this data, we present a hybrid learned dynamics model that combines the known physics-based RK4 dynamics with a residual network to model the full nonlinear quadrotor dynamics. Our work shows that this hybrid model can then be utilized by solvers such as CasADi and IPOPT to solve a nonlinear MPC problem and obtain an NMPC controller for the system. Through our results, we show that the controller achieves the desired performance on our target hover task, validating the approach.

To certify the stability of the controller around the hover point, we develop a generalized Lyapunov function using the framework of [1], with a quadratic base inherited from our NMPC terminal cost and a learned residual correction term. Our results show that 99.80% of the trajectories in the operating domain satisfy the generalized decrease condition, and the Lyapunov function shape and level-set analysis further support the claim. These results indicate that our pipeline produces NMPC controllers whose closed-loop stability can be empirically certified using learned Lyapunov functions.

A. Milestones, Contributions, and Code

From our results, we also conclude that we satisfy all the core milestones proposed in the introduction. Our rollout results, plots, and supplementary code depict a working simulation environment from which rollout data can be obtained, meeting our first milestone. The MPC performance validates our learned dynamics model, meeting the second milestone. The performance plots show that the obtained NMPC controller achieves the desired tracking and stabilization behavior, meeting the third milestone. Our in-class demonstration and the supplementary code present a closed-loop simulation of the validated controller running the hover stabilization task, meeting the fourth milestone. Finally, the Lyapunov analysis results show that our approach yields a valid certificate over the stability region, meeting the fifth milestone. The stretch milestone of collecting rollouts on a physical drone could not be completed due to hardware unavailability in our partner research lab.

In this project, Aniket was responsible for the NMPC controller and the Lyapunov certification development. Avi

was responsible for the simulation environment setup and rollout data collection.

The code for the project, along with instructions for reproducing the plots shown above, is available at:
<https://github.com/AviDube/MPC-and-Lyapunov-for-drones>.

REFERENCES

- [1] K. Long, J. Cortés, and N. Atanasov, “Certifying Stability of Reinforcement Learning Policies using Generalized Lyapunov Functions,” arXiv preprint arXiv:2505.10947, 2025. doi: 10.48550/arXiv.2505.10947.
- [2] Z. Chen and L. Kehler, “A model predictive control approach for quadrotor cruise control,” arXiv preprint arXiv:2504.13286v1, Apr. 2025. doi: 10.48550/arXiv.2504.13286.